# Automatización con Bash y Python

**Sistemas operativos**

**06/03/2026**

**Autores :**

Naela Khaldi
Zineb Hamman

**Curso Académico: 2025/2026**

# INDICE :

1. Introduccion : 3
2. Objectivos: 3
3. Marco teorico: 3
4. Diseno y metodología: 5
5. Implementacion: 6
6. Resultados y pruebas : 7
7. Discusion: 7
8. Conclusion : 8
9. Manual de ejecucion: 8
10. Repositorio git: 9
11. Bibliografia: 10

## 1. Introduccion :

En el ambito de los sistemas operativos, la automatización y la gestión de procesos son habilidades fundamentales.Esta practica se centra en ampliar la funcionalidad de un software heredado que captura datos de un sensor utilizando el protocolo MQTT. El programa original presenta varias limitaciones, ya que únicamente muestra los datos de los sensores en terminal y no dispone de mecanismos para almacenar la información . Como consecuencia, el sistema no permite mantener un registro de los datos capturados ni facilita su análisis posterior o su explotación.

Para solucionar estas limitaciones, se desarrollo una solución utilizando Bash y Python. El sistema automatiza la captura de datos, gestiona correctamente los procesos del sistema, almacena la salida en archivos de registro (log), analiza datos estructurados en formato JSON y genera graficas en formato png y representaciones ASCII directamente en terminal.

## 2. Objectivos:

Los principales objetivos de esta practica son:

- Ejecutar un programa desde Bash y capturar su salida.
- Redirigir correctamente la salida estándar (stdout) y el flujo de errores (stderr) hacia un archivo de registro (.log)
- Gestionar procesos del sistema utilizando su identificador de proceso (PID).
- Verificar si un proceso esta activo mediante el uso del comando kill -0.
- Utilizar señales del sistema operativo para finalizar procesos (SIGINT,SIGTERM,SIGKILL).
- Ejecutar código Python desde un Bash script.
- Extraer datos en formato JSON.
- Generar representaciones graficas de los datos en formato PNG.
- Mostrar visualizaciones de datos en la terminal mediante grafica ASCII.

## 3. Marco teorico:

Los principales conceptos teoricos implicados en esta practica son los siguientes:

- **Proceso:** representa una instancia de un programa en ejecución dentro del sistema operativo.Cada proceso posee su propio espacio de memoria y recursos asignados por el sistema.

- **PID (Process ID):** un identificador único asignado por el sistema operativo a cada proceso en ejecución. Este identificador permite gestionar y controlar los procesos desde la línea de comandos o mediante scripts. Gracias al PID es posible comprobar si un proceso sigue activo, enviar señales para modificar su comportamiento o finalizar su ejecución.

- **Senales del sistema operativo:** mecanismos utilizados por el sistema operativo como Linux para la comunicación y control de procesos. En esta practica se utilizan señales como SIGINT,SIGTERM,SIGKILL para detener o finalizar procesos de ejecución, se utilizan de forma progresiva para intentar cerrar el proceso de forma segura antes de forzar su terminación.

- **Redireccion de flujos estándar:** Permite dirigir la salida de un programa hacia dieferentes destinos como un archivo .Mediante operadores como > para redirigir la salida estándar (stdout) y 2>&1 para regirigir también la salida de errores(stderr) al mismo archivo. Esto permite almacenar toda la información generada por el programa en log , facilitando su análisis posterior.

- **Automatizacion mediante scripts:** permite ejecutar tareas de forma automática sin intervención manual. En este caso, un script en Bash se encarga de ejecutar el programa MQTT, capturar su salida , controlar el tiempo de ejecución y lanzar posterioremente el script de análisis en Python.

- **Parseo de JSON:** El proceso de interpretar y extraer información estrecturada desde datos en formato JSON. Este dormato es utilizado para el intercambio de datos debido a su estructura clara y fácil d eprocesar. En esta practica se analizan los datos recibidos desde los sensores para convertirlos en estructuras de datos utilizables dentro del programa en Python.

- **Visualizacion científica:** Represnetar datos mediante 2 tipos de visualizaciones : graficas en formato PNG generadas con la librería matplotllib, y por otro lado, una grafica ASCII que permite mostrar una representación visual de los datos directamente en la terminal utilizando caracteres de texto.

- **MQTT :** un protocolo de mensajería ligero. Funciona mediante un modelo publicador-suscriptor, donde los sensore spublican información en un bróker MQTT y otros programas pueden susribirse para recibir esos datos. En esta practica , el software se conecta a un bróker MQTT y recibe datos provenientes de sensores como sensor/data/sen55 y sensor/data/gas_sensor . Estos datos son enviados en formato JSON y capturados posterioremente por el sistema desarrollado mediante Bash y analizados utilizando Python.

## 4. Diseno y metodología:

El sistema esta compuesto por tres elementos principlaes.

-MQTT:

Elprograma se ecarga de contectarse al bróker MQTT y revivir los datos enviados por los sensores.Estos datos incluyen diferentes mediciones .El programa muestra información directamente en ele terminal, pero no dispone de funcionalidades para almacenar los datos ni para analizarlos posterioremente.

- Script Bash (capture_mqtt.sh):

  El script Bash se encarga de automatizar el proceso de captura de datos. En primer lugar, solicita al usuario el tiempo de captura en segundos. Posteriormente ejecuta el programa MQTT en segundo plano y redirige su salida al archivo mqtt_capture.log, donde se almacenan los datos recibidos. El script obtiene el identificador del proceso (PID) mediante la variable $!, lo que permite controlar el proceso en ejecución. Durante el tiempo indicado, el script verifica periódicamente que el proceso sigue activo utilizando el comando kill -0. Una vez finalizado el tiempo de captura, el script termina el proceso mediante señales del sistema operativo.

-Script Python (plots_mqtt.py):

Se encarga de procesar el archivo de registro generado durante la captura de datos. El programa lee el archivo mqtt_capture.log, identifica las líneas

que contienen información de sensores y extrae los datos en formato JSON.
A partir e estos datos se obtienen los valores relevante .Posterioremente el programa genera graficos en formato PNG utilizando la librería matplotlib y también muestra una representación grafica en formato ASCII directamente en la terminal.
+ Herramientas utilizadas para el desarrollo del sistema son:
-Bash : para la automatización de procesos y gestión de la ejecucion del programa MQTT.
-Python: para el procesamiento de datos y generación de visualizaciones.
-Matplotlib: para la creación de graficas en formato PNG.
-MQTT: protocolo de comunicación utilizado por los sensores.

## 5. Implementacion:

La implementación del sistema se divide en dos componentes : un script en Bash , encargado de la captura y control del proceso , y un script en Python , encargado del análisis y visualización de los datos.

**Bash script:**

El script Bash solicita al usuario el tiempo de captura en segundos. Si el usuario no introduce un valor, el sistema establece un tiempo por defecto de 10 segundos**.**
Posteriormente, el script ejecuta el programa mqtt_subscribe_emqx_linux en segundo plano y redirige su salida al archivo mqtt_capture.log, almacenando tanto la salida estándar como los errores. Después de iniciar el programa, el script obtiene el PID del proceso mediante la variable $!.

Durante el tiempo de captura, el script verifica si el proceso sigue activo utilizando el comando kill -0. Cuando se alcanza el tiempo máximo, el proceso se finaliza enviando señales del sistema operativo (SIGINT**,** SIGTERM **y** SIGKILL**)**.

Finalmente, el script ejecuta un ejemplo de código Python dentro de Bash y lanza el programa plots_mqtt.py, que se encarga de analizar los datos capturados.

**Python script:**

El script Python abre el archivo mqtt_capture.log y busca las líneas que contienen Payload, donde se encuentran los datos de los sensores en formato JSON. Estos datos se procesan utilizando la librería json para extraer los valores numéricos.

A partir de estos valores se genera un gráfico utilizando Matplotlib, El gráfico muestra la evolución de los datos capturados y se guarda automáticamente en la carpeta plots en formato PNG con el nombre data.png. Si la carpeta no existe, el programa la crea automáticamente utilizando la función os.makedirs.

Además, el script genera una gráfica ASCII en la terminal, permitiendo visualizar los datos directamente en consola sin necesidad de abrir el archivo externos.

## 6. Resultados y pruebas :

Para verificar el correcto funcionamiento del sistema, se ejecuta el script principal mediante el comando: ./capture_mqtt.sh

Durante la ejecución, el sistema realiza correctamente las siguientes acciones:

- Inicia el programa MQTT y comienza la captura de datos de los sensores.
- Guarda la salida del programa en el archivo mqtt_capture.log.
- Finaliza el proceso automáticamente una vez alcanzado el tiempo de captura especificado.
- Ejecuta el script de análisis en Python.
- Genera automáticamente la carpeta plots/ si no existe.
- Produce un gráfico en formato PNG (data.png) con los valores capturados.
- Muestra una gráfica ASCII en la terminal que permite visualizar los datos directamente desde la consola.
  Para comprobar la estabilidad del sistema, se realizaron varias pruebas utilizando diferentes tiempos de captura, como 10, 40 y 60 segundos. En todos los casos el sistema funcionó correctamente, capturando los datos, generando el archivo log y produciendo las visualizaciones correspondientes.

## 7. Discusion:

La solución desarrollada cumple con los requisitos de la práctica. El sistema permite capturar los datos generados por el programa MQTT, guardarlos en un archivo de registro y analizarlos posteriormente mediante un script en Python. Todo el proceso funciona sin modificar el programa

original, lo que demuestra que es posible ampliar las funcionalidades de un software utilizando scripts para automatizar tareas.

Sin embargo, el sistema presenta algunas limitaciones. El análisis de los datos depende de la estructura del JSON recibido en el campo **Payload**, por lo que cambios en el formato de los datos podrían afectar al funcionamiento del script. Además, el programa necesita la librería **Matplotlib** para generar los gráficos, por lo que esta dependencia debe estar instalada previamente.

En el futuro, el sistema podría mejorarse añadiendo soporte para diferentes tipos de sensores y estructuras de datos. También sería posible almacenar la información en una base de datos para facilitar su análisis a largo plazo. Finalmente, se podría mejorar el control de errores y la validación de los datos para hacer el sistema más robusto.

## 8. Conclusion :

Esta practica demuestra como es posible integrar diferentes herramientas como Bash y Python , para ampliar las funcionalidades de un software existente.Mediante el uso de srcipts, se logro automatizar la captura de datos generados por el programa MQTT y realizar su posterior análisis y visualización.

Durante el desarrollo de la practica se aplicaron varios conceptos importantes de la asignatura, como la gestión de procesos mediante el PID, el uso de señales del sistema, la redirección de flujos estándar y la automatización de tareas mediante scripts.

Ademas , se aplicaron técnicas de procesamiento de datos en Python , oncluyendo el análisis de datos en formato JSON y la generación de graficos para visualizar la información obtenida de los sensores .
Se cumplieron los objetivos planteados en la práctica, desarrollando un sistema que permite capturar, analizar y visualizar datos de sensores de forma automática

## 9. Manual de ejecucion:

Para ejecutar el sistema es necesario disponer de los siguientes elementos:

- Sistema operativo Ubuntu Linux o un entorno compatible con Linux.
- Python 3 instalado en el sistema.
- La librería Matplotlib para la generación de gráficos.

⇨ La librería Matplotlib puede instalarse mediante el siguiente comando:

```
pip3 install matplotlib
```

Pasos de ejecución:

1. Dar permisos de ejecución al script Bash:

```
chmod +x capture_mqtt.sh
```

2. Ejecutar el sistema con el siguiente comando:

```
./capture_mqtt.sh
```

3. El programa solicitará al usuario el tiempo de captura en segundos.

4. Una vez finalizada la ejecución, el sistema generará automáticamente:

- El archivo de registro mqtt_capture.log con los datos capturados.
- La carpeta plots/ que contiene el gráfico generado en formato PNG**.**
- Una gráfica ASCII que se mostrará directamente en la terminal.

## 10. Repositorio git:

https://github.com/thecoolestray/Practica_1_SO.git

https://github.com/naelakh/Practica_1_SO.git

```
Practica_1_SO
│
├── Ejecutables/
├── plots/
├── scripts/
├── src/
├── capture_mqtt.sh
├── plots_mqtt.py
├── mqtt_capture.log
└── README.md
```

## Explicacion  del contenido:

**Ejecutables:**
Carpeta que contiene el programa ejecutable utilizado para conecta  al broker MQTT y recibir los datos enviados por los sensores. Este programa es el encargado de mostrar los datos en la terminal.

**Plots:**
Directorio donde se almacenan las gráficas generadas automaticamente por el script python. En esta carpeta se guardan las imagenes en formato PNG creadas a partir de los datos capturados.

**Scripts:**
Carpeta destinada a almacenar scripts auxiliares relacionados con la ejecución o automatización del sistema.

**Src:**
Directorio que contiene archivo de código fuente del programa original utilizado para conectarse al broker MQTT y recibir los datos de los sensores.

**capture_mqtt.sh:**
Script Bash encargado de ejecutar el programa MQTT, controlar el tiempo de captura de datos, gestionar el proceso mediante su PID y guardar la salida en el archivo mqtt_capture.log.

**plots_mqtt.py:**
Script Python responsable de analizar el archivo de registro, extraer los datos en formato JSON y generar las visualizaciones gráficas en formato PNG y ASCII.

**mqtt_capture.log:**
Archivo de registro que almacena los datos capturados durante la ejecución del programa MQTT.

**README.md :**
Archivo de documentación del proyecto que explica la estructura del repositorio y los pasos necesarios para ejecutar el sistema.

## 11. Bibliografia:


- Universidad de Burgos. *Material de la asignatura Sistemas Operativos en UBUVirtual*.
- GitHub. *Documentación sobre estructura de  README , informe y repositorio*: https://github.com/jarain78/Practica_0_SO.git

- Eclipse Paho MQTT Project
- Json encoder and decoder
- Libreria Matplotlib